

Devops Build App — Application Deployment Report

Implementation & Verification Report

Docker · DockerHub · AWS EC2 · Jenkins (Multibranch Pipeline) · Prometheus · Grafana · Alertmanager · GitHub

Prepared By	Mohana Srinivasan (mohandevopssme@gmail.com)
Date	27 July 2026
AWS Account / Region	074925547344 / us-east-1
GitHub Repository	github.com/Mohan6201/guvi_tasks (path: Project-3/devops-build; branches: dev, main)
Upstream App Repo	github.com/sriram-R-krishnan/devops-build (ships build/ output only)
DockerHub Images	mohan809/dev (Public) · mohan809/prod (Private)
Jenkins Job	devops-build-pipeline (Multibranch Pipeline), branches dev & main tracked independently
Deployed Application	http://18.212.58.111 (app t2.micro – terminated after verification, see Section 19)
Monitoring	Prometheus + Grafana ("Node Exporter Full" dashboard) + Alertmanager, real email alerts verified

This document records the deployment of the devops-build React application — a pre-built frontend shipped upstream as a build/ folder with no source — to a production AWS environment via Docker, a Jenkins Multibranch Pipeline, and DockerHub branch-routed image repositories. Two EC2 t2.micro instances were provisioned via plain AWS CLI bash scripts (no Terraform, per this project's own requirements): one running the live application and the monitoring stack, one running Jenkins. The report follows the actual order of execution — the real commands run, the real configuration files, and every real issue hit and fixed along the way, including several that only surfaced once things were actually running against live AWS/DockerHub/GitHub/Jenkins APIs rather than by reading the configuration. All AWS infrastructure was terminated after verification (Section 19) to avoid ongoing charges.

CONTENTS

1. Objective
2. Prerequisites & Environment
3. Configuration — .env & Project Structure
4. Application Source & Local Docker Verification
5. DockerHub Registry
6. Version Control
7. AWS — App Instance Provisioning
8. First Manual Deploy & Live Verification
9. AWS — Jenkins Instance Provisioning
10. Jenkins Installation & Setup Wizard
11. Jenkins Plugins & Credentials
12. Jenkins Pipeline Job — Multibranch Pipeline
13. GitHub Webhook Integration
14. Declarative Pipeline — Jenkinsfile
15. Pipeline Execution — Build History & Issues Fixed
16. Live Application Verification
17. Monitoring Setup — Prometheus, Grafana, Alertmanager
18. Monitoring Verification — Dashboards & Real Alerts
19. Infrastructure Teardown
20. Submission Summary

1. Objective

Deploy the devops-build React application (a pre-built frontend, shipped upstream with no source — only a `build/` folder) to a production-ready state on AWS. The task requirements: a Dockerfile and docker-compose.yml, exactly two bash scripts (`build.sh`, `deploy.sh`), version control on GitHub with CLI-only git commands and a dev branch, two DockerHub repositories (dev public, prod private) with branch-based routing, a Jenkins pipeline that auto-triggers from both dev and master/main and routes images to the matching DockerHub repo, one AWS t2.micro instance running the application with a security group open on port 80 to everyone and port 22 restricted to the operator's own IP, and open-source monitoring with a down-notification. No Terraform, Kubernetes, or EKS are named anywhere in this project's requirements, so the infrastructure here is plain AWS CLI bash scripting throughout.

2. Prerequisites & Environment

All work was performed from Git Bash on Windows, in `Project-3/devops-build` (a subfolder of the guvi_tasks monorepo), against AWS account 074925547344 in us-east-1. AWS CLI identity was confirmed before provisioning anything:

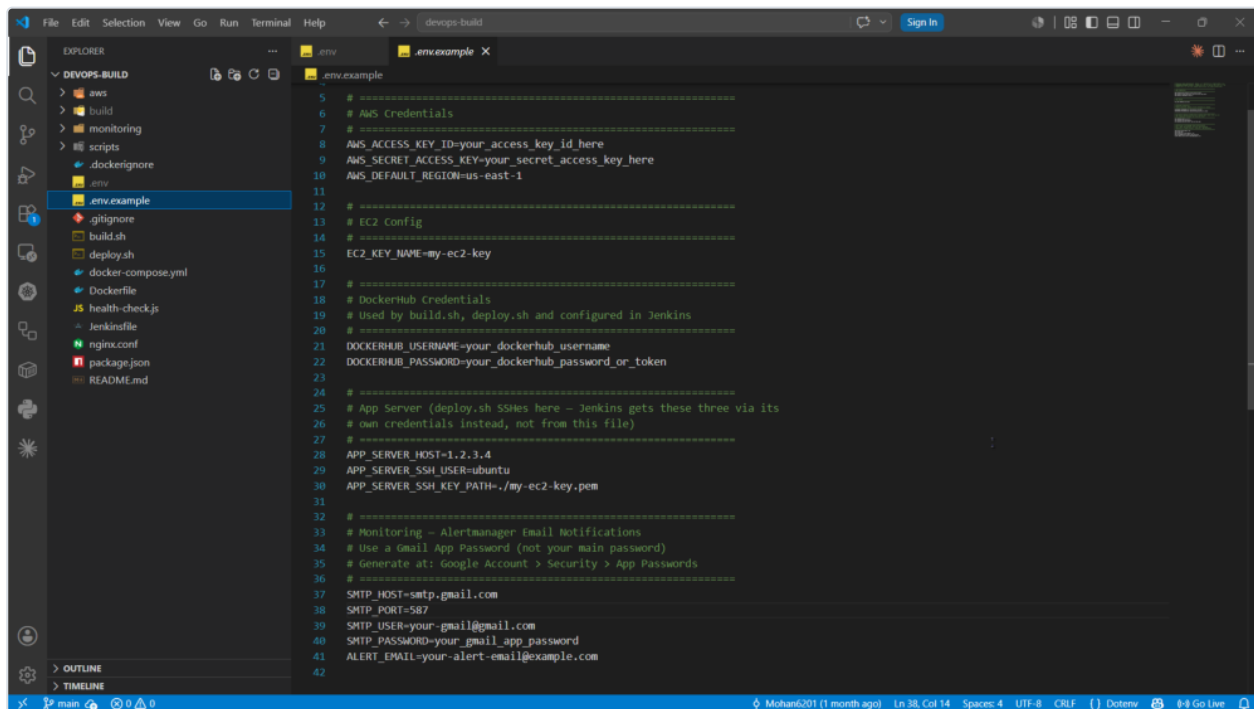
COMMANDS EXECUTED

```
$ aws sts get-caller-identity
{
  "UserId": "AIDARC4PJD5IHY4BBYGSM",
  "Account": "074925547344",
  "Arn": "arn:aws:iam::074925547344:user/mohana_srinivasan"
}
```

Windows' native `ssh.exe` (used from a plain cmd.exe window later in this project) and Git Bash's bundled SSH client behave differently around private-key file permission checks — this distinction mattered more than once later in the deployment (Section 17) and is called out where it came up.

3. Configuration — .env & Project Structure

A single `.env` file (gitignored, based on a tracked `.env.example` template) drives every script — AWS region/key name, DockerHub credentials, app-server SSH details, and SMTP credentials for alerting. No AWS access keys or secrets are ever hardcoded in any script; everything reads from environment variables.



Screenshot 1 — Project structure in VS Code alongside `.env.example`, showing every credential category (AWS, EC2 key, DockerHub, app server, SMTP/Alertmanager) driven from one file.

4. Application Source & Local Docker Verification

The upstream repository (sriram-R-krishnan/devops-build) ships only a pre-built `build/` folder — confirmed byte-identical to this project's copy via a direct git tree hash comparison against the GitHub API before anything was built. The `Dockerfile` is a minimal `nginx:alpine` image serving that folder on port 80, with a custom `nginx.conf` exposing `/health` and `/metrics` for the monitoring stack built later (Section 17).

COMMANDS EXECUTED

```
$ set -a; source .env; set +a
$ docker build -t devops-build-app .
[+] Building 3.8s (10/10) FINISHED
=> [1/4] FROM docker.io/library/nginx:alpine
=> CACHED [2/4] RUN rm /etc/nginx/conf.d/default.conf
=> CACHED [3/4] COPY nginx.conf /etc/nginx/conf.d/
=> CACHED [4/4] COPY build/ /usr/share/nginx/html/

$ docker run -d --name local-test -p 80:80 devops-build-app
docker: Error response from daemon: Conflict. The container name "/local-test" is already in use ...
```

⚠ Issue — container name conflict on first local run

A leftover container from an earlier test still held the name `local-test`, so the fresh `docker run` was rejected outright.

✓ Fixed

COMMANDS EXECUTED

```
$ docker rm -f local-test
$ docker run -d --name local-test -p 80:80 devops-build-app
$ curl http://localhost/health
healthy
```

```
MINGW64 ~/Users/mgnor/OneDrive/Desktop/Tenila/guvf_tasks/Project-3/devops-build
mgnor@MGNOR MINGW64 ~/OneDrive/Desktop/Tenila/guvf_tasks/Project-3/devops-build (main)
$ aws sts get-caller-identity
{
  "UserId": "AIDANCPJ05JHVB8BVGDM",
  "Account": "074925347344",
  "Arn": "arn:aws:iam::074925347344:user/mohana_sriniwasan"
}

mgnor@MGNOR MINGW64 ~/OneDrive/Desktop/Tenila/guvf_tasks/Project-3/devops-build (main)
$ export $(cat .env | grep -v '#' | xargs)
mgnor@MGNOR MINGW64 ~/OneDrive/Desktop/Tenila/guvf_tasks/Project-3/devops-build (main)
$ set -a
mgnor@MGNOR MINGW64 ~/OneDrive/Desktop/Tenila/guvf_tasks/Project-3/devops-build (main)
$ source .env
mgnor@MGNOR MINGW64 ~/OneDrive/Desktop/Tenila/guvf_tasks/Project-3/devops-build (main)
$ set +a

mgnor@MGNOR MINGW64 ~/OneDrive/Desktop/Tenila/guvf_tasks/Project-3/devops-build (main)
$ docker build -t devops-build-app .
[+] Building 1.4s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> [internal] load metadata for docker.io/library/nginx:alpine 0.0s
=> [auth] library/nginx:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore 0.0s
=> [internal] load context: 5638 0.0s
=> [2/4] FROM docker.io/library/nginx:alpine@sha256:4a73073bd537c65b739505da037898461f3beebc3c2c3aaac22d2af70c1732 0.0s
=> [internal] load build context 0.0s
=> transferring context: 8678 0.0s
=> CACHED [2/4] RUN rm /etc/nginx/conf.d/default.conf 0.0s
=> CACHED [3/4] COPY nginx.conf /etc/nginx/conf.d/ 0.0s
=> CACHED [4/4] COPY build/ /usr/share/nginx/html/ 0.0s
=> exporting to image 0.0s
=> exporting layers 0.0s
=> writing image sha256:750840b67f227b83cd92e7cde8ff7c3ff63c4f702f28274a3b8e4f7cd9d49e 0.0s
=> naming to docker.io/library/devops-build-app 0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/A2Fax7ldbpsilytbnv8Zrw

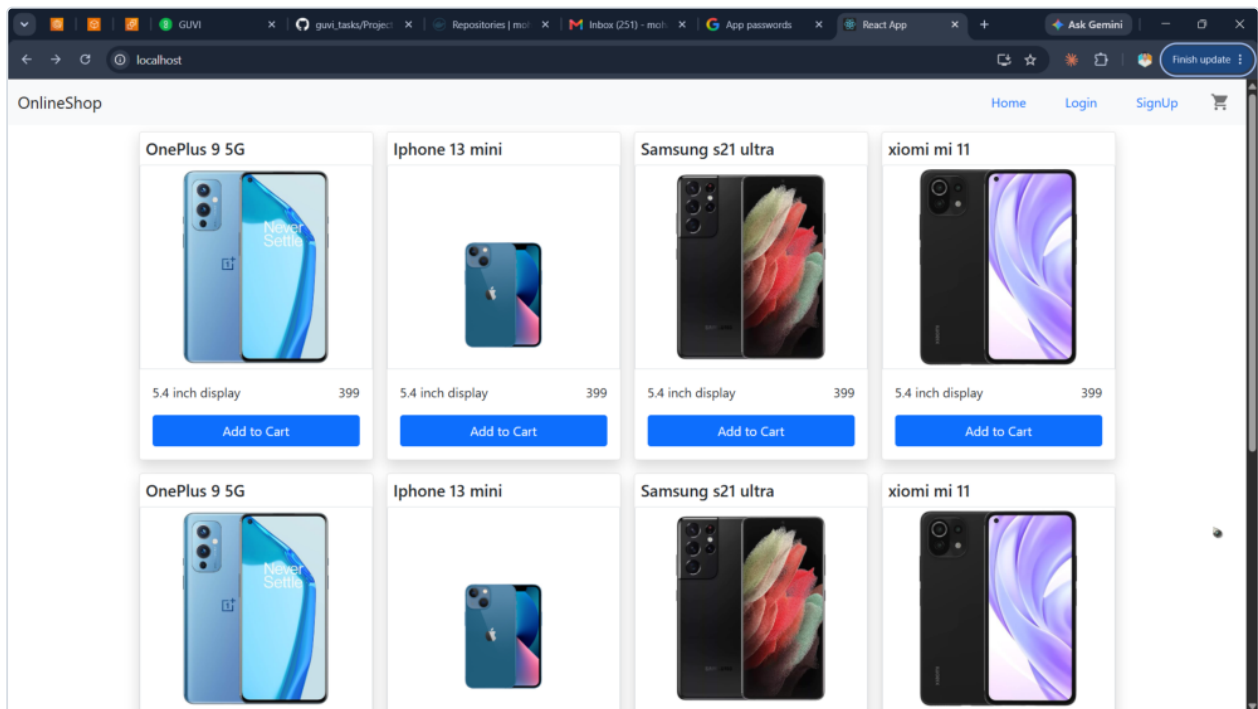
What's next:
View a summary of image vulnerabilities and recommendations - docker scout quickview

mgnor@MGNOR MINGW64 ~/OneDrive/Desktop/Tenila/guvf_tasks/Project-3/devops-build (main)
$ docker run -d --name local-test -p 80:80 devops-build-app
docker: Error response from daemon: Conflict. The container name "/local-test" is already in use by container "2efee9458ea93c374512d990ebbf9cfdai56fc23e4ba920b14db18194abdc". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information

mgnor@MGNOR MINGW64 ~/OneDrive/Desktop/Tenila/guvf_tasks/Project-3/devops-build (main)
$
mgnor@MGNOR MINGW64 ~/OneDrive/Desktop/Tenila/guvf_tasks/Project-3/devops-build (main)
$ docker rm -f local-test
local-test
mgnor@MGNOR MINGW64 ~/OneDrive/Desktop/Tenila/guvf_tasks/Project-3/devops-build (main)
$ docker run -d --name local-test -p 80:80 devops-build-app
174ae30d5ed73fa6aa0d2db60d7556d458cd824da7121b937145b647faaa49
mgnor@MGNOR MINGW64 ~/OneDrive/Desktop/Tenila/guvf_tasks/Project-3/devops-build (main)
$ curl http://localhost/health
healthy
```

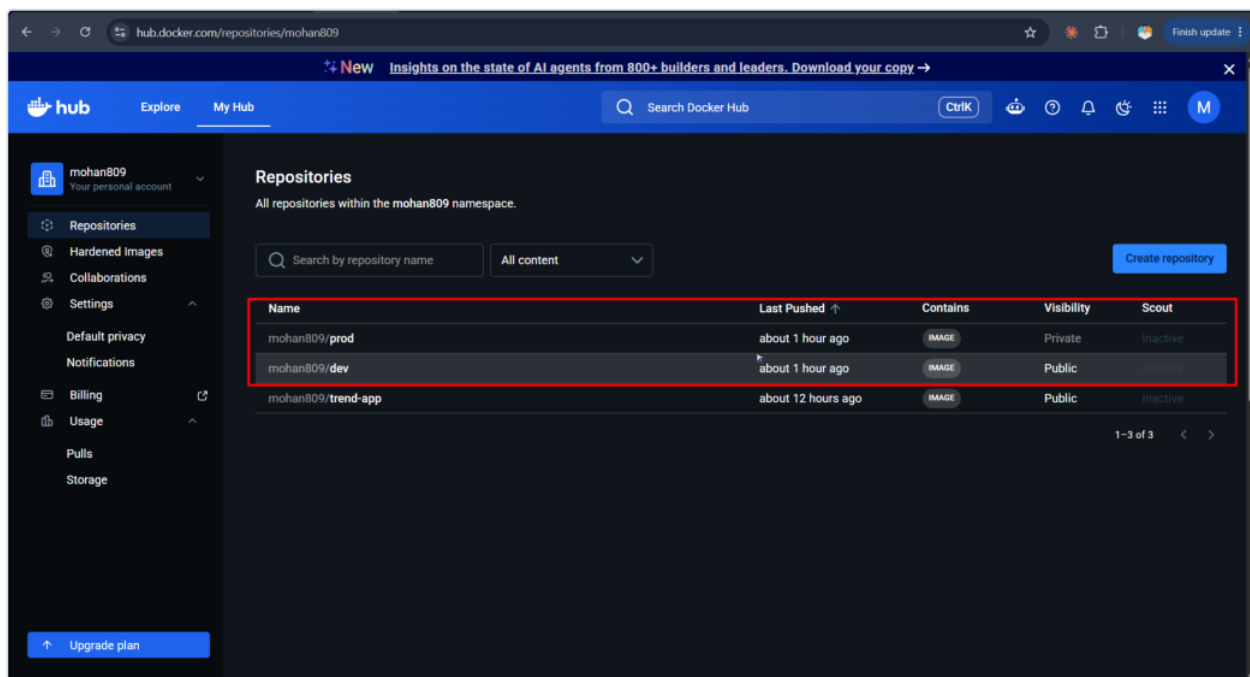
Screenshot 2 — Local build, the container-name conflict and its fix, and curl confirming /health returns healthy.



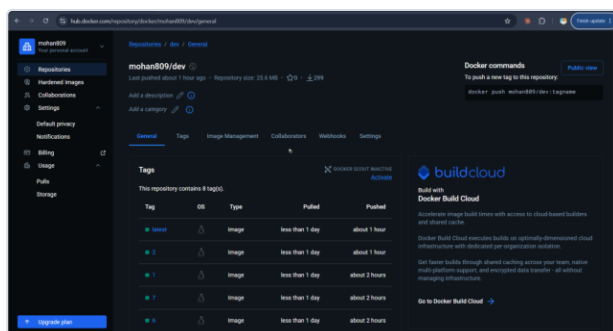
Screenshot 3 — The application (OnlineShop) rendering correctly at localhost, product grid loading with live images from an external product-data API the app calls at runtime.

5. DockerHub Registry

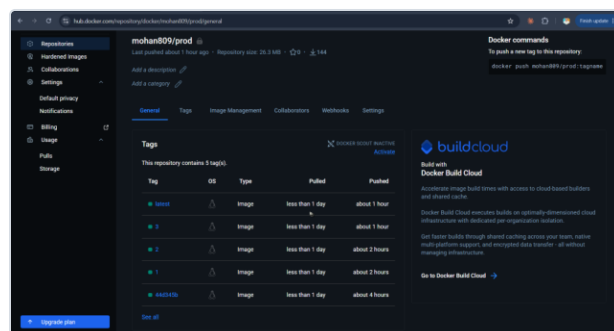
Two repositories under the mohan809 namespace back the branch-routed image push: dev (public) and prod (private). Both are auto-created by DockerHub on first docker push from the pipeline rather than pre-created via the UI — auto-created repositories default to public visibility, which meant the prod repository briefly needed a manual visibility correction after its first push to actually satisfy the "prod must be private" requirement. That correction is reflected in the final state shown below.



Screenshot 4 — Final repository list under mohan809: prod (Private) and dev (Public), both actively pushed to by the pipeline.



Screenshot 5 — mohan809/dev repository detail, public, with real pushed tags from successive Jenkins builds.



Screenshot 6 — mohan809/prod repository detail — lock icon confirms private visibility, real tags from the main-branch pipeline.

6. Version Control

The codebase lives in the Mohan6201/guvi_tasks GitHub monorepo under Project-3/devops-build, on both main (this monorepo's default branch, standing in for the spec's master — build.sh, deploy.sh, and the Jenkinsfile all treat the two as equivalent) and dev. All git operations throughout this project were performed via CLI only, per the explicit requirement — no GitHub web-upload UI was used at any point in this session. .gitignore / .dockerignore exclude the usual local artifacts while deliberately tracking build/, since it is this project's actual application output, not a regenerable build artifact.

7. AWS — App Instance Provisioning

The application runs on a single t2.micro instance, provisioned by `aws/provision-app.sh` — plain AWS CLI, no Terraform. Its security group matches the spec literally: port 80 open to 0.0.0.0/0, port 22 restricted to the operator's own IP. Two real bugs surfaced on the first live run and were fixed in the script itself before the second, successful run shown in the same session:

COMMANDS EXECUTED

```
$ ./provision-app.sh
Provisioning app instance in us-east-1
Your IP: 223.178.81.231
Using AMI: ami-0d001f8052688dc45

An error occurred (InvalidParameterValue) when calling the CreateSecurityGroup operation:
Value
(App instance – port 80 public, SSH restricted to operator IP) for parameter
GroupDescription is
invalid. Character sets beyond ASCII are not supported.
```

⚠ Issue — security-group description used a non-ASCII em dash

AWS's `GroupDescription` parameter is ASCII-only, and the description text used an em dash character.

✓ Fixed

COMMANDS EXECUTED

```
$ ./provision-app.sh
...
Security Group: sg-062bc4be91d48e187
Key pair saved as my-ec2-key.pem

Error parsing parameter '--user-data': Unable to load paramfile file:///c:/Users/.../
scripts/setup_app.sh:
[Errno 2] No such file or directory
```

⚠ Issue — Windows path resolution failure on --user-data

`aws.exe` is a native Windows binary and cannot resolve the POSIX-style path (`/c/Users/...`) Git Bash builds for a `file://` reference.

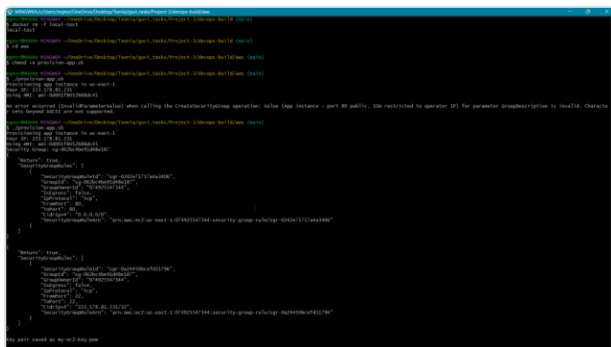
✓ Fixed

Passed the user-data script's content directly (`--user-data "$(cat scripts/setup_app.sh)"`) instead of a file reference — this sidesteps the path-resolution problem entirely and works identically on any OS. The security-group creation step was also made idempotent (check-then-create) so a script that fails partway through can be safely re-run without hitting a duplicate-resource error, exercised in the third run below:

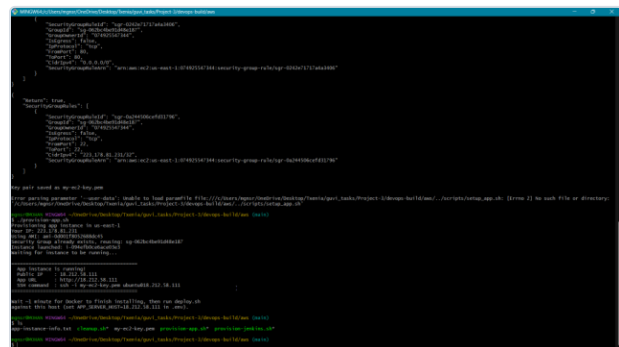
COMMANDS EXECUTED

```
$ ./provision-app.sh
Provisioning app instance in us-east-1
Your IP: 223.178.81.231
Using AMI: ami-0d001f8052688dc45
Security Group already exists, reusing: sg-062bc4be91d48e187
Instance launched: i-094efb0ce6ace03e3
Waiting for instance to be running...

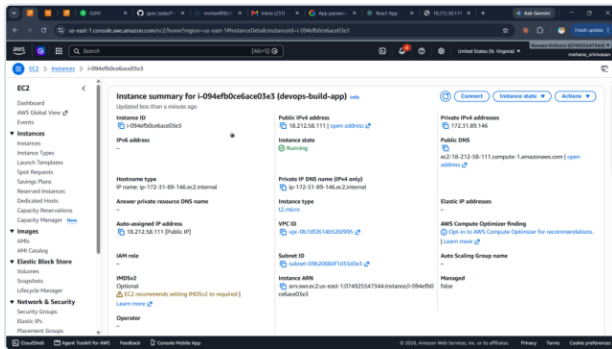
=====
App instance is running!
Public IP      : 18.212.58.111
App URL       : http://18.212.58.111
SSH command   : ssh -i my-ec2-key.pem ubuntu@18.212.58.111
=====
```



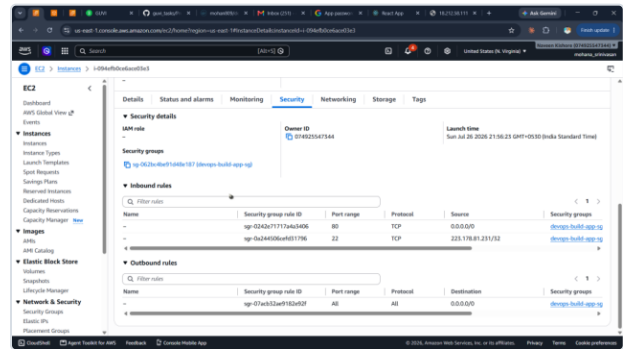
Screenshot 7 — The ASCII em-dash failure and the successful retry immediately after, security group and key pair created.



Screenshot 8 — The Windows path-resolution failure on `--user-data`, then the fixed script running clean end to end: instance launched and running at 18.212.58.111.



Screenshot 9 — AWS EC2 console: devops-build-app instance summary, Running, t2.micro, public IP 18.212.58.111.



Screenshot 10 — Security group inbound rules: port 80 from 0.0.0.0/0, port 22 from the operator's IP only — matching the spec exactly.

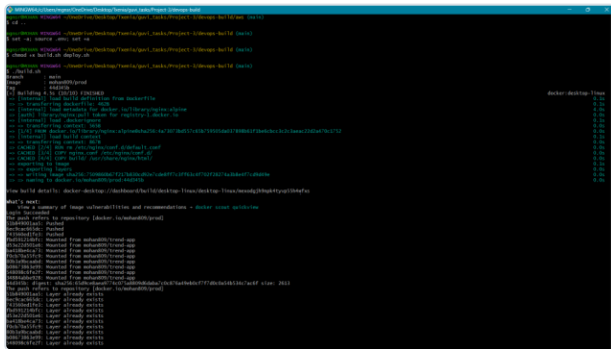
8. First Manual Deploy & Live Verification

Before any Jenkins automation existed, build.sh and deploy.sh were run manually once to prove the deploy path itself works end to end — build, push to DockerHub, SSH to the app server, pull, and run.

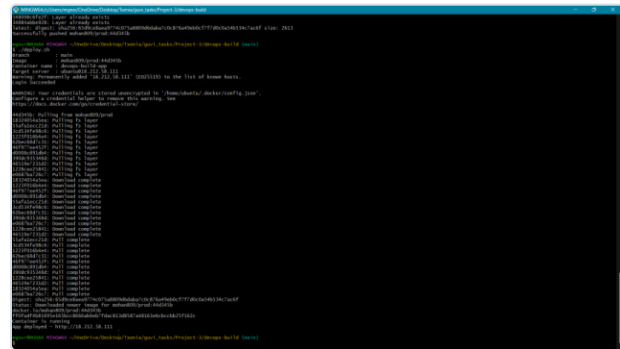
COMMANDS EXECUTED

```
$ ./build.sh
Branch      : main
Image       : mohan809/prod
Tag         : 44d345b
...
Successfully pushed mohan809/prod:44d345b

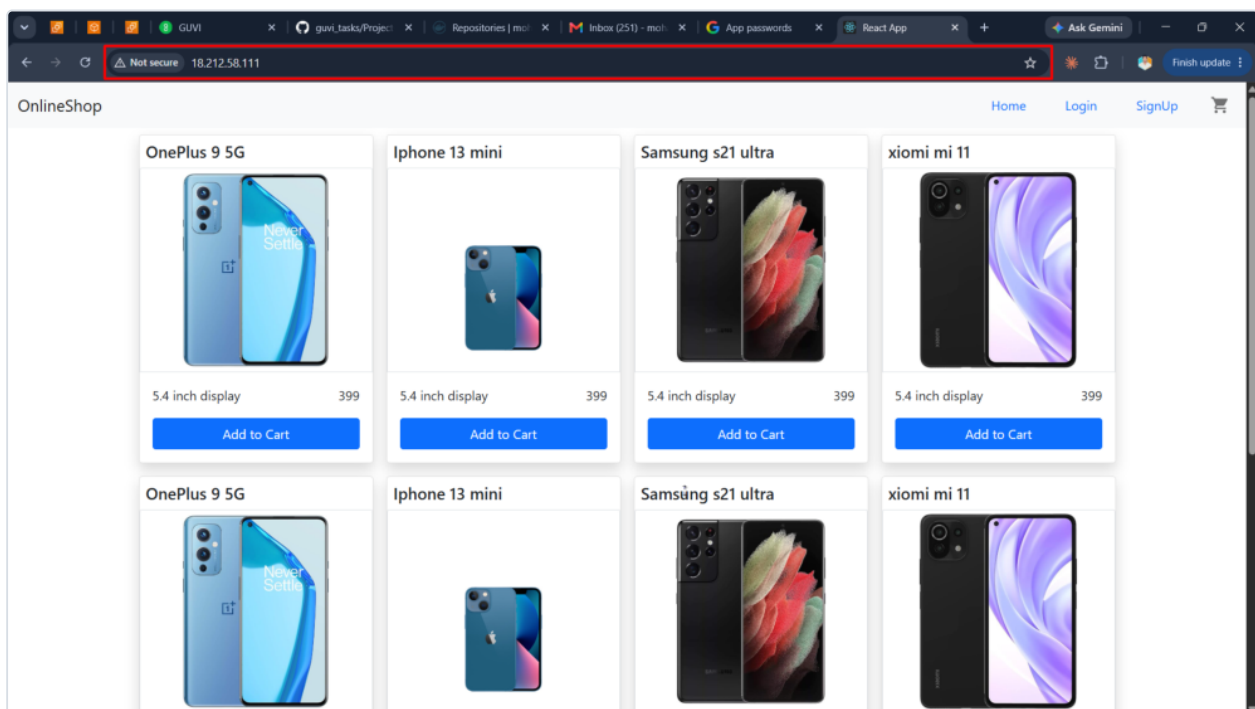
$ ./deploy.sh
Branch      : main
Image       : mohan809/prod:44d345b
Container name : devops-build-app
Target server  : ubuntu@18.212.58.111
...
Container is running
App deployed — http://18.212.58.111
```



Screenshot 11 — Manual build.sh run on main, correctly routing to the prod image per the branch-detection logic.



Screenshot 12 — deploy.sh pulling the image on the app server and starting the container — "Container is running".



Screenshot 13 — The application live in a browser at <http://18.212.58.111>, confirming the whole build-push-deploy path works before any CI automation is layered on top.

9. AWS — Jenkins Instance Provisioning

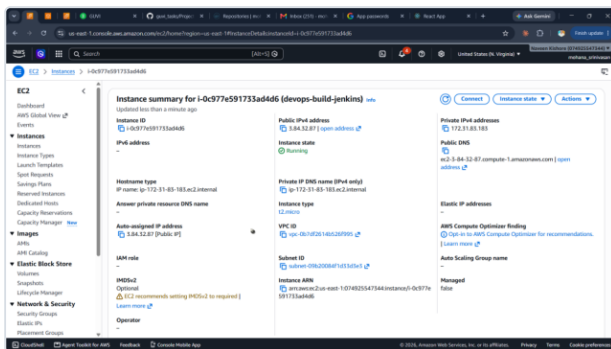
Jenkins runs on its own separate t2.micro — the spec only names one instance for the app, but a single t2.micro is too tight to run Jenkins, Docker builds, and a live container simultaneously, so

Jenkins gets a second instance with its own security group (port 8080 open for the UI and GitHub webhook delivery, port 22 restricted to the operator's IP).

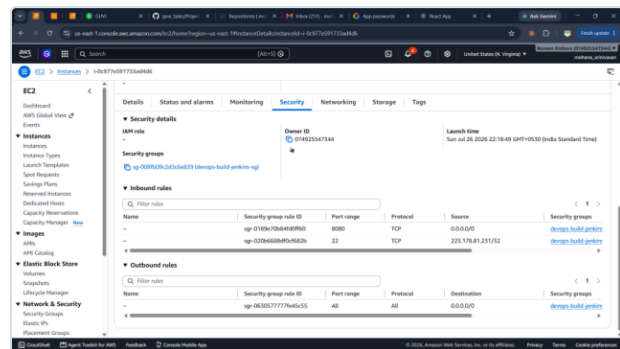
COMMANDS EXECUTED

```
$ ./provision-jenkins.sh
Provisioning Jenkins instance in us-east-1
Your IP: 223.178.81.231
Security Group: sg-008fb09c2d3c6e839
Instance launched: i-0c977e591733ad4d6
Waiting for instance to be running...
```

```
=====
Jenkins instance is running!
Public IP    : 3.84.32.87
Jenkins URL  : http://3.84.32.87:8080
=====
```



Screenshot 14 — AWS EC2 console: devops-build-jenkins instance summary, Running, public IP 3.84.32.87.



Screenshot 15 — Jenkins instance's security group: port 8080 public, port 22 restricted to the operator's IP.

10. Jenkins Installation & Setup Wizard

Jenkins and Docker are installed via the instance's user-data script. The very first automated run hit a real, live failure that had nothing to do with this project's own code: the Jenkins apt signing key had rotated upstream. Docker installed and started cleanly, but apt-get update for the Jenkins repository failed with NO_PUBKEY, and because the script uses set -e, everything after that line — including the actual apt-get install jenkins — was silently skipped.

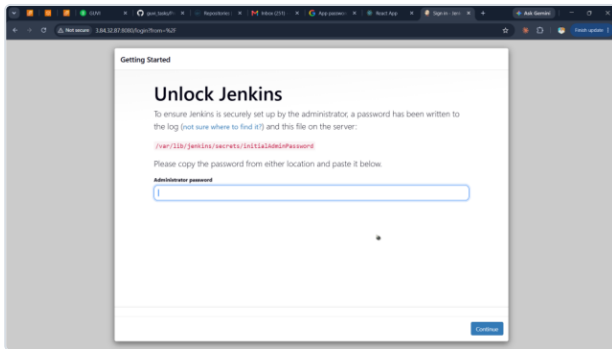
⚠ Issue — Jenkins apt signing key expired

The old key URL (jenkins.io-2023.key) still returned HTTP 200, but its key fingerprint (5BA31D57EF5975CA) no longer matched what actually signs the live Jenkins repository (7198F4B714ABFC68) — confirmed by downloading both and comparing fingerprints with `gpg --show-keys`.

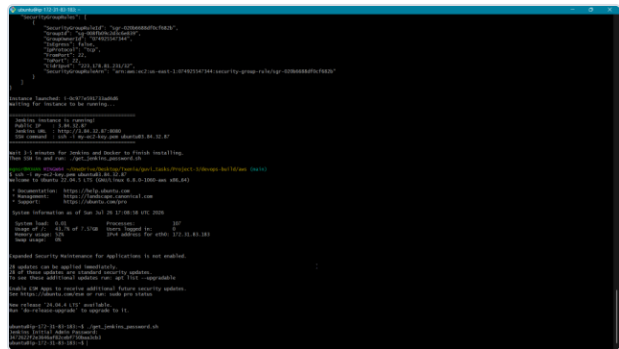
✓ Fixed

The current correct key (jenkins.io-2026.key) was identified from Jenkins' own live install documentation, its fingerprint verified to match the missing key exactly before changing anything, and the script fixed (also bumping the keyring path to the now-standard `/etc/apt/keyrings/` location). The already-running instance was patched in place over SSH rather than relaunched, and the fix was committed back into `scripts/setup_jenkins.sh` for any future provisioning.

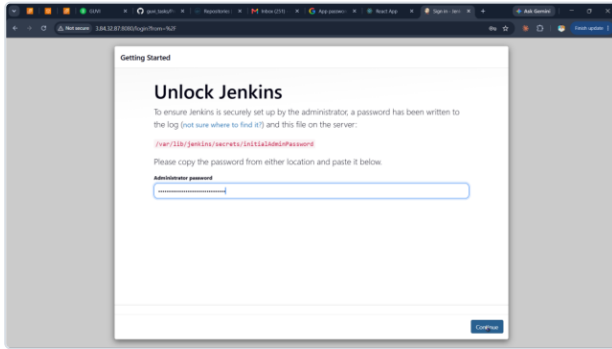
With that fixed, the standard setup wizard completed cleanly: unlock with the initial admin password, install suggested plugins, create the first admin user, confirm the instance URL.



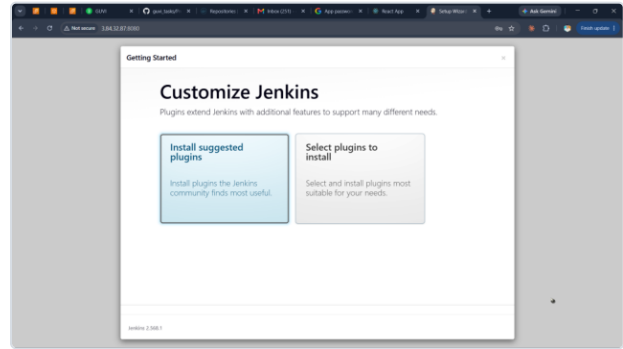
Screenshot 16 — "Unlock Jenkins" — the initial setup wizard, confirming Jenkins actually came up after the signing-key fix.



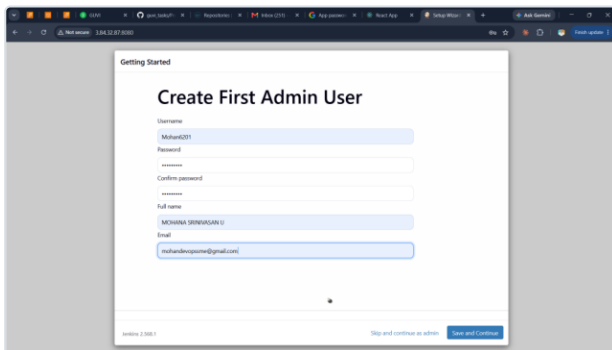
Screenshot 17 — Retrieving the initial admin password over SSH via `get_jenkins_password.sh`.



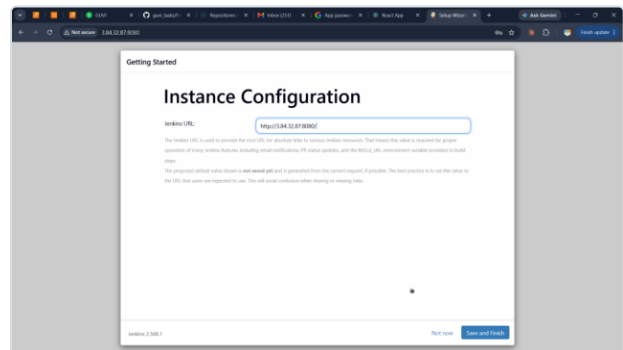
Screenshot 18 — Password entered on the unlock screen.



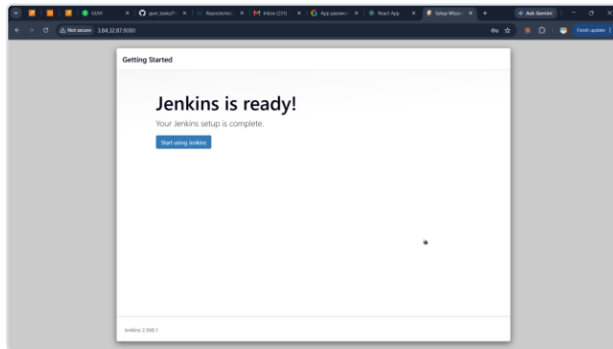
Screenshot 19 — "Install suggested plugins" selected.



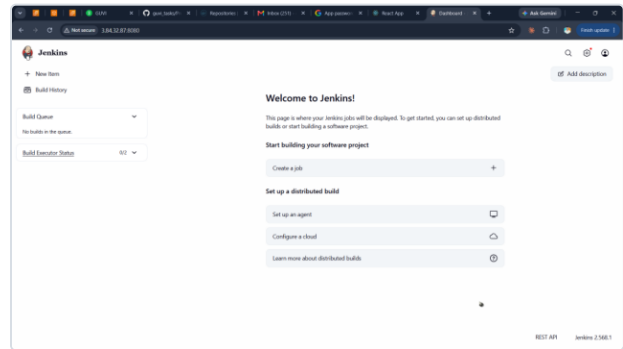
Screenshot 20 — First admin user created (Mohan6201).



Screenshot 21 — Instance configuration confirming the Jenkins URL.



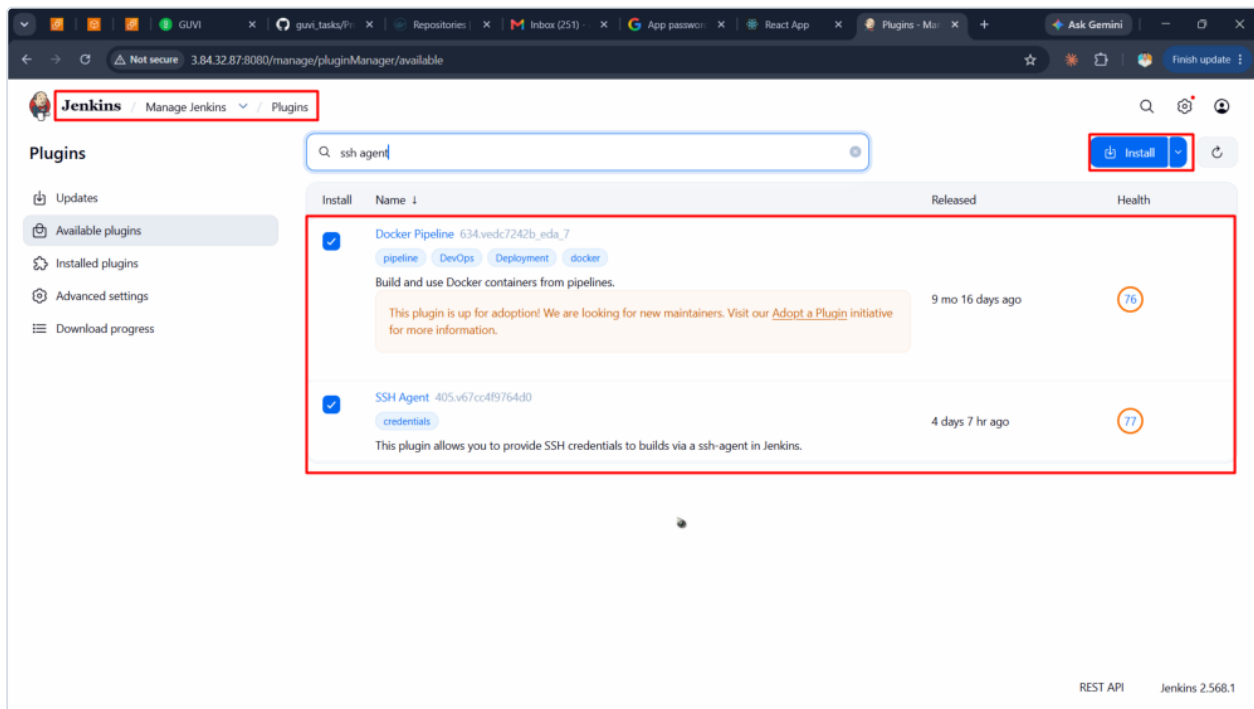
Screenshot 22 — "Jenkins is ready!"



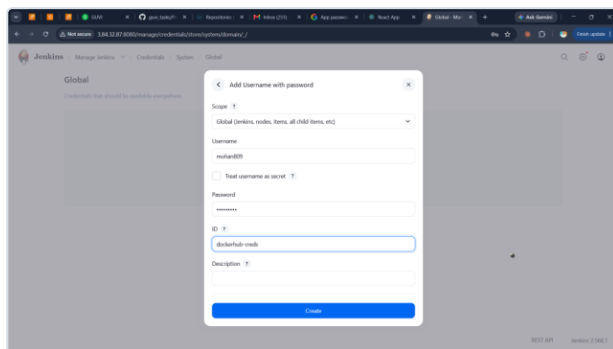
Screenshot 23 — Jenkins dashboard, ready to configure the pipeline.

11. Jenkins Plugins & Credentials

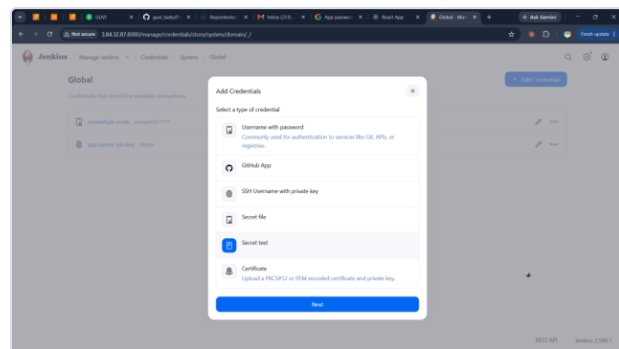
Two additional plugins were installed beyond the suggested set: **Docker Pipeline** (to build/push images from the pipeline) and **SSH Agent** (to provide the SSH credential binding the Deploy stage uses). Three credentials were then configured: a DockerHub username/password (dockerhub-creds), an SSH private key for the app server (app-server-ssh-key), and the app server's host address as a secret text value (app-server-host).



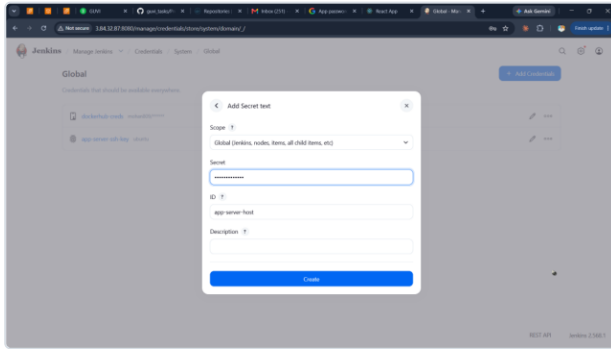
Screenshot 24 — Installing the Docker Pipeline and SSH Agent plugins.



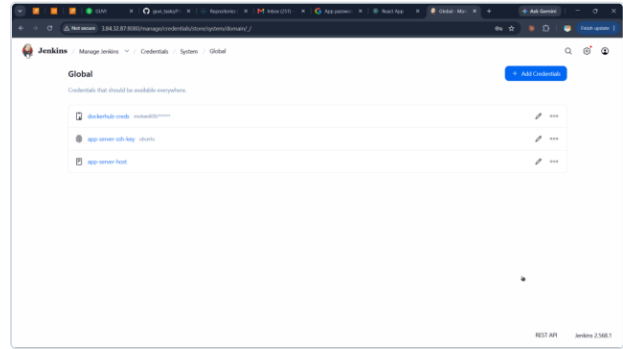
Screenshot 25 — Adding the DockerHub username/password credential, ID dockerhub-creds — must match the Jenkinsfile exactly.



Screenshot 26 — SSH credential app-server-ssh-key successfully added to the global credential store.



Screenshot 27 — Adding app-server-host as a Secret text credential.



Screenshot 28 — All three global credentials listed: dockerhub-creds, app-server-ssh-key, app-server-host.

The app server's security group was also extended at this point to trust SSH connections specifically from the Jenkins instance's security group (not opened to the internet — a security-group-referenced rule, so only an instance that is itself a member of the Jenkins security group can reach port 22 on the app server). This was necessary because the Deploy stage runs from the Jenkins host, not the operator's own machine, and the original port-22 rule only trusted the operator's IP.

12. Jenkins Pipeline Job — Multibranch Pipeline

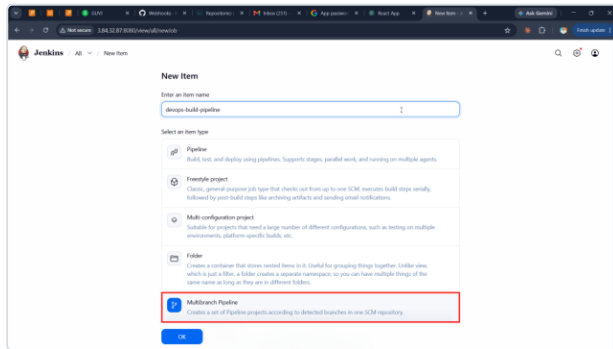
The job was initially created as a plain "Pipeline script from SCM" job with two hardcoded branch specs (* /dev, * /main) and a GitHub push trigger. This worked for dev — every push to dev correctly auto-triggered a build — but a real push to main, confirmed delivered successfully by GitHub's own webhook log (200 OK, correct refs/heads/main ref), never triggered a build at all.

⚠ Issue — Multibranch limitation of the plain Pipeline job type

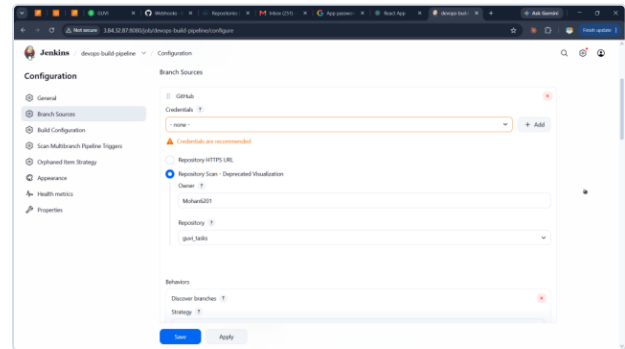
With multiple hardcoded branch specs, Jenkins' lightweight-checkout polling only reliably tracks change-detection for one of them. A manual "Build Now" always worked (it just builds whatever the specs currently resolve to, bypassing polling entirely), which is exactly why the gap wasn't obvious until an automatic trigger was specifically tested.

✓ Fixed

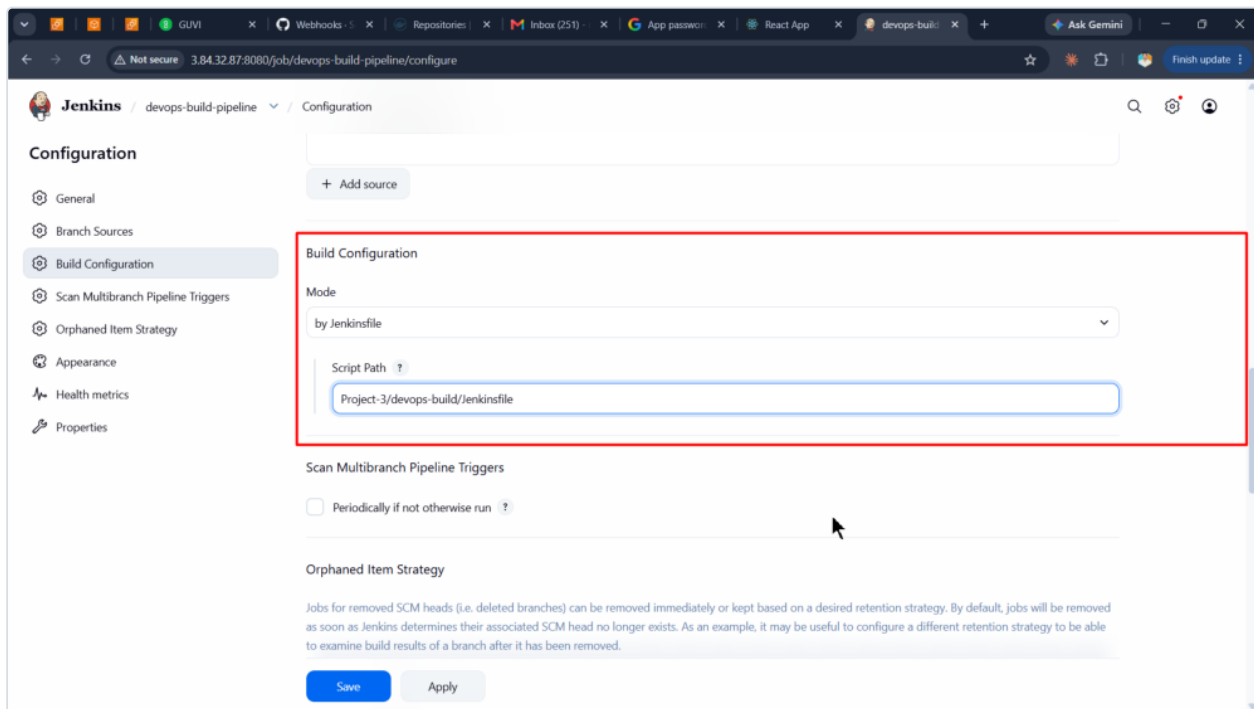
Recreated the job as a **Multibranch Pipeline** — the job type Jenkins actually provides for "discover and independently track multiple branches, auto-trigger each on its own push," backed by the GitHub Branch Source plugin (installed for this), which reuses the same webhook endpoint already configured. The Jenkinsfile's branch-detection logic was also updated to prefer the Multibranch-native `env.BRANCH_NAME` variable, falling back to `env.GIT_BRANCH` for compatibility.



Screenshot 29 — Creating devops-build-pipeline as a Multibranch Pipeline.



Screenshot 30 — Branch source configured against Mohan6201/guvi_tasks.



Screenshot 31 — Build configuration: mode "by Jenkinsfile", Script Path Project-3/devops-build/Jenkinsfile.

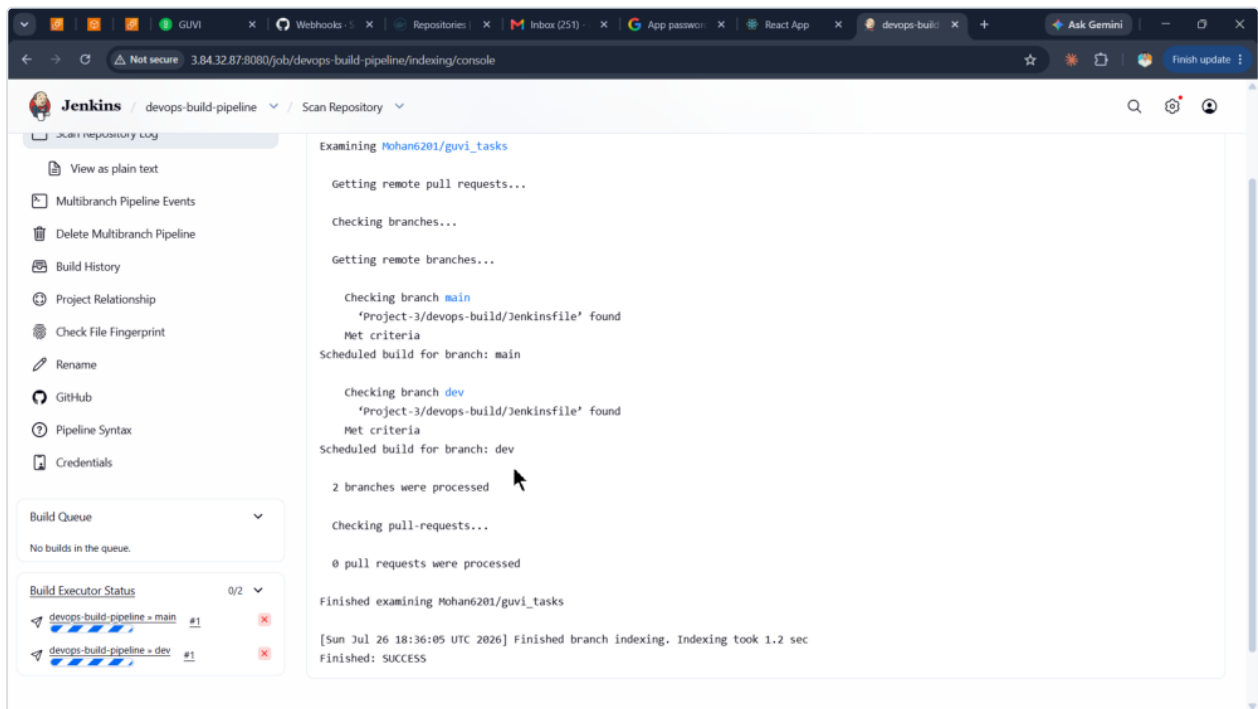
Saving triggers an immediate first-time branch scan. Anonymous GitHub API access is limited to 60 requests/hour, and Jenkins' own rate limiter deliberately throttled the very first scan to stay under that budget — visible as several minutes of "Sleeping" in the scan log rather than an actual error. Adding a GitHub Personal Access Token as a credential (5,000 requests/hour once authenticated) removed the throttling for all subsequent scans. The scan itself discovered both branches and correctly scheduled a build for each independently:

COMMANDS EXECUTED

```
Checking branch main
'Project-3/devops-build/Jenkinsfile' found
Met criteria
Scheduled build for branch: main

Checking branch dev
'Project-3/devops-build/Jenkinsfile' found
Met criteria
Scheduled build for branch: dev

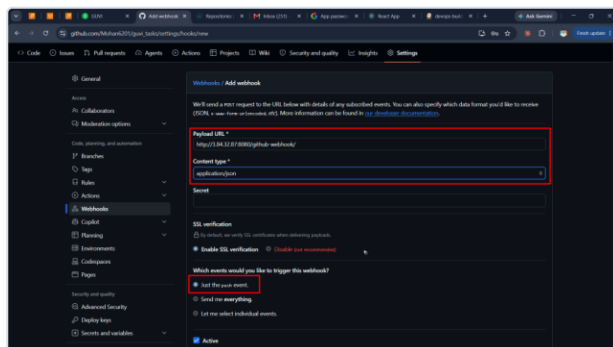
2 branches were processed
Finished: SUCCESS
```



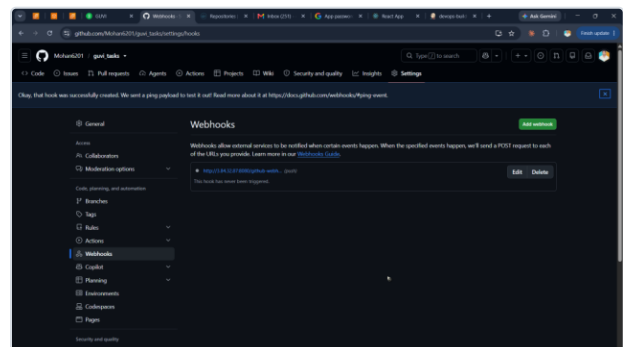
Screenshot 32 — Repository scan log: both main and dev discovered, a build scheduled for each independently.

13. GitHub Webhook Integration

A single push-event webhook on Mohan6201/guvi_tasks targets the Jenkins instance directly. It did not need to change when the job was recreated as Multibranch — the GitHub Branch Source plugin listens on the same `/github-webhook/` endpoint the classic trigger used, it just does more with each delivery (resolve which specific branch changed and trigger only that branch's build, instead of one shared trigger for the whole job).



Screenshot 33 — Webhook configuration: payload URL `http://3.84.32.87:8080/github-webhook/`, content type `application/json`, push event only.



Screenshot 34 — GitHub confirms the webhook was created successfully.

14. Declarative Pipeline — Jenkinsfile

Four stages: Checkout, Determine Target Repo (branch-based routing to dev or prod), Build Docker Image, Push to DockerHub, Deploy. checkout scm pulls the entire guvi_tasks monorepo into the workspace root — the job's Script Path setting only tells Jenkins where to find this Jenkinsfile, it does

not change the working directory for the pipeline's own steps — so every shell step is wrapped in `dir("Project-3/devops-build")`, confirmed necessary by a real first build failure (Section 15).

JENKINSFILE (CORE STAGES, AS COMMITTED TO THE REPOSITORY)

```
environment {
    DOCKERHUB_CREDS      = credentials('dockerhub-creds')
    DOCKERHUB_USERNAME    = "${DOCKERHUB_CREDS_USR}"
    DOCKERHUB_PASSWORD    = "${DOCKERHUB_CREDS_PSW}"
    IMAGE_TAG             = "${env.BUILD_NUMBER}"
    APP_DIR               = "Project-3/devops-build"
}

stage('Determine Target Repo') {
    script {
        def branch = env.BRANCH_NAME ?: env.GIT_BRANCH?.replaceAll('origin/', '') ?: 'dev'
        env.TARGET_REPO = (branch == 'master' || branch == 'main')
            ? "${DOCKERHUB_USERNAME}/prod" : "${DOCKERHUB_USERNAME}/dev"
    }
}

stage('Build Docker Image') {
    dir(APP_DIR) {
        sh 'docker build -t ${TARGET_REPO}:${IMAGE_TAG} . && docker tag ${TARGET_REPO}:${IMAGE_TAG} ${TARGET_REPO}:latest'
    }
}

stage('Deploy') {
    dir(APP_DIR) {
        withCredentials([
            sshUserPrivateKey(credentialsId: 'app-server-ssh-key', keyFileVariable:
'APP_SERVER_SSH_KEY_PATH', usernameVariable: 'APP_SERVER_SSH_USER'),
            string(credentialsId: 'app-server-host', variable: 'APP_SERVER_HOST')
        ]) {
            sh 'chmod +x deploy.sh && ./deploy.sh'
        }
    }
}
```

deploy.sh itself SSHes from the Jenkins host into the app server and runs the pulled image there — "deploying" always means a remote operation, since Jenkins and the app never share a host. A container-name race between concurrent branch deploys (Section 15) was also fixed here by collapsing a separate stop-then-remove into a single atomic `docker rm -f`.

15. Pipeline Execution — Build History & Issues Fixed

Several real, live build failures occurred before both branches ran cleanly end to end — each fixed by committing a corrected file, not by working around it.

⚠ Issue — Build #1 (main) — Dockerfile not found

COMMANDS EXECUTED

```
+ docker build -t mohan809/prod:1 .  
ERROR: failed to build: failed to solve: failed to read dockerfile: open Dockerfile: no  
such file or directory
```

✓ Fixed

Confirmed the root cause described in Section 14 — the checkout pulls the whole monorepo, and the build step was running from the workspace root instead of Project-3/devops-build/. Fixed by wrapping the Build/ Push/Deploy stages in dir(APP_DIR).

⚠ Issue — Deploy stage — SSH connection timeout (twice, two different causes)

COMMANDS EXECUTED

```
+ ./deploy.sh  
Target server : ubuntu@****  
ssh: connect to host **** port 22: Connection timed out
```

✓ Fixed

First cause: the app server's security group only trusted the operator's own IP for SSH — Jenkins, running on a separate instance, was never authorized. Fixed by adding the security-group-referenced rule described in Section 11. **Second cause,** after that fix still timed out: same-VPC instances connecting to each other's *public* IP do not reliably match security-group-source rules — the connection round-trips through the Internet Gateway instead of staying on the internal network. Confirmed by testing raw TCP connectivity to both the public and private IP from inside the Jenkins instance — the private IP connected instantly, the public IP still timed out with identical security group rules either way. The app-server-host credential was updated to the app instance's private IP; the operator's own local .env correctly keeps the public IP, since a laptop outside the VPC has no private-network path to the instance at all.

⚠ Issue — Deploy stage — container name conflict

COMMANDS EXECUTED

```
docker: Error response from daemon: Conflict. The container name "/devops-build-app" is  
already  
in use by container "...". You have to remove (or rename) that container to be able to  
reuse that name.
```

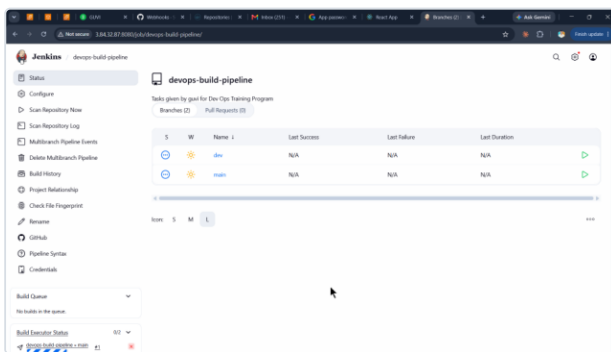
✓ Fixed

This happened twice: once when the Multibranch job's first-time scan triggered both branches close enough together that their Deploy stages raced on the same container name, and once when a routine git push to dev (to sync an unrelated fix) coincided with a manual retrigger of main — confirmed by checking the actual Jenkins build folders on disk over SSH, which showed a dev build running at the identical timestamp as the main build that failed. deploy.sh's separate docker stop + docker rm (each swallowing errors with || true) left a race window; collapsed to a single atomic `docker rm -f`, and re-running once the concurrent build had settled succeeded cleanly both times.

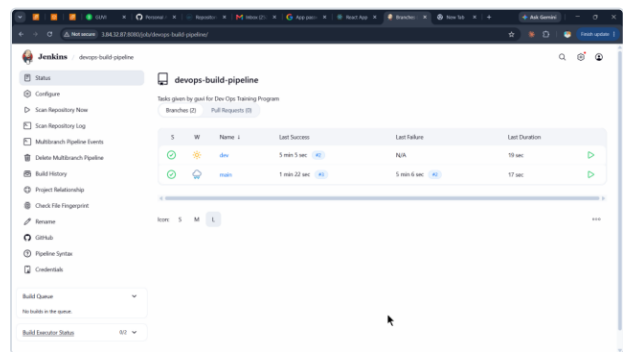
Both branches ultimately built, pushed, and deployed successfully:

COMMANDS EXECUTED

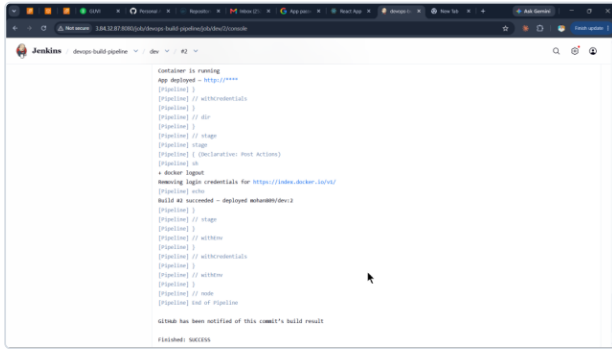
```
Branch: dev -> Target repo: mohan809/dev  
Build #2 succeeded - deployed mohan809/dev:2  
  
Branch: main -> Target repo: mohan809/prod  
Build #3 succeeded - deployed mohan809/prod:3
```



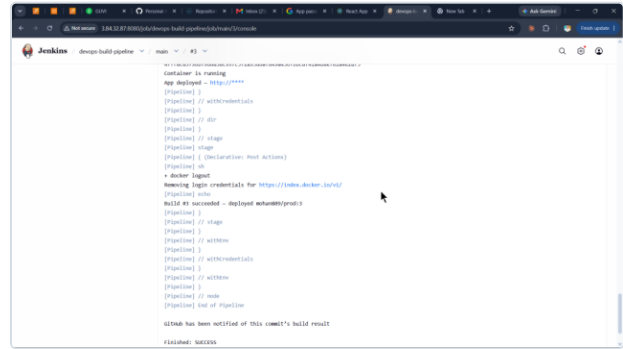
Screenshot 35 — Both branches discovered as independent sub-jobs under the Multibranch Pipeline.



Screenshot 36 — Final state: dev and main both green, with main's build history showing the two earlier real failures before the fix.



Screenshot 37 — dev build #2 console output: "Build #2 succeeded — deployed mohan809/dev:2", "Finished: SUCCESS".



Screenshot 38 — main build #3 console output: "Build #3 succeeded — deployed mohan809/prod:3", "Finished: SUCCESS".

16. Live Application Verification

After the pipeline settled, the app server was confirmed running the correct, latest image with a direct check over SSH and a plain HTTP request — not just a green Jenkins build:

COMMANDS EXECUTED

```
$ docker ps --format "{{.Names}}\t{{.Image}}\t{{.Status}}"
devops-build-app mohan809/dev:2 Up 2 minutes

$ curl -s -o /dev/null -w "%{http_code}\n" http://18.212.58.111/
200
```

17. Monitoring Setup — Prometheus, Grafana, Alertmanager

The monitoring stack (Prometheus, Grafana, Alertmanager, node-exporter, all via docker-compose) runs on the app instance itself, keeping total infrastructure at the two instances actually named in the spec. Rather than opening additional public ports for Grafana/Prometheus/Alertmanager on the app security group, an SSH tunnel through the already-open, already-restricted port 22 was used to view the dashboards locally — avoiding widening the security group beyond what the spec asks for.

⚠ Issue — docker-compose CLI not installed; Windows SSH key permissions rejected

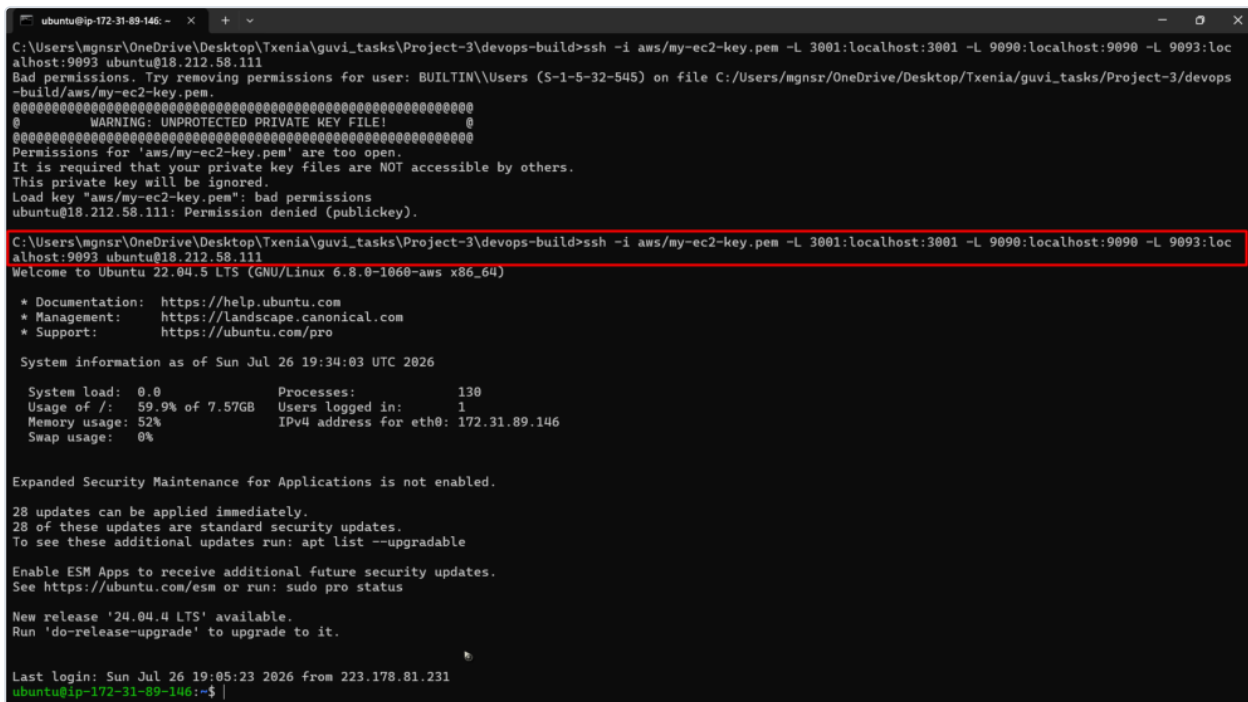
Two real, live-only bugs surfaced once this stack was actually started on the real Ubuntu host, neither visible from reading the config: `docker-compose` (the legacy hyphenated CLI) is not installed by this Docker install — modern installs ship Compose as a plugin, invoked as `docker compose` (no hyphen); and the Windows `ssh.exe` client used to open the tunnel initially refused the private key with a "bad permissions" / "UNPROTECTED PRIVATE KEY FILE" error, since its Windows ACL still granted read access to the broader `BUILTIN\Users` group.

✓ Fixed

Used `docker compose` (no hyphen) throughout, and fixed the key permissions with `icacls` to restrict the file to the current user only.

COMMANDS EXECUTED

```
$ cd monitoring
$ ./render-config.sh
Rendered alertmanager.yml.template -> alertmanager.yml
$ docker compose up -d
[+] up 4/4
Container alertmanager      Running
Container node-exporter     Running
Container prometheus        Running
Container grafana           Running
```



```
C:\Users\mgnsr\OneDrive\Desktop\Txenia\guvi_tasks\Project-3\devops-build>ssh -i aws/my-ec2-key.pem -L 3001:localhost:3001 -L 9090:localhost:9090 -L 9093:localhost:9093 ubuntu@18.212.58.111
Bad permissions. Try removing permissions for user: BUILTIN\Users (S-1-5-32-545) on file C:/Users/mgnsr/OneDrive/Desktop/Txenia/guvi_tasks/Project-3/devops-build/aws/my-ec2-key.pem.
WARNING: UNPROTECTED PRIVATE KEY FILE!
Permissions for 'aws/my-ec2-key.pem' are too open.
It is required that your private key files are NOT accessible by others.
This private key will be ignored.
Load key "aws/my-ec2-key.pem": bad permissions
ubuntu@18.212.58.111: Permission denied (publickey).

C:\Users\mgnsr\OneDrive\Desktop\Txenia\guvi_tasks\Project-3\devops-build>ssh -i aws/my-ec2-key.pem -L 3001:localhost:3001 -L 9090:localhost:9090 -L 9093:localhost:9093 ubuntu@18.212.58.111
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1060-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun Jul 26 19:34:03 UTC 2026

System load:  0.0          Processes:    130
Usage of /:   59.9% of 7.57GB   Users logged in: 1
Memory usage: 52%          IPv4 address for eth0: 172.31.89.146
Swap usage:  0%

Expanded Security Maintenance for Applications is not enabled.

28 updates can be applied immediately.
28 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

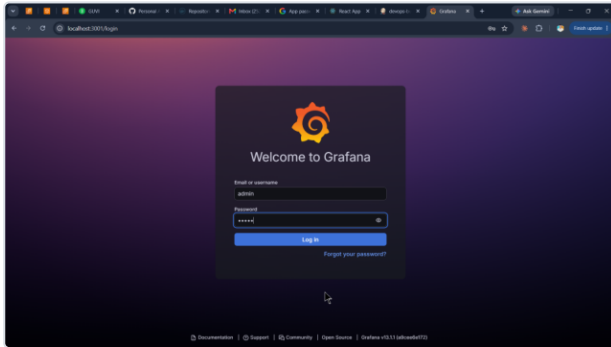
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

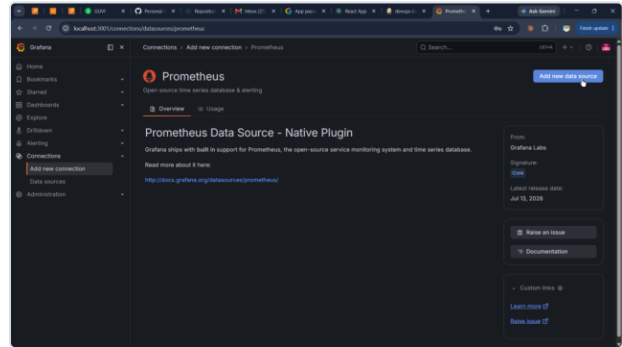
Last login: Sun Jul 26 19:05:23 2026 from 223.178.81.231
ubuntu@ip-172-31-89-146:~$
```

Screenshot 39 — The Windows SSH "bad permissions" failure and the corrected tunnel connecting successfully immediately after.

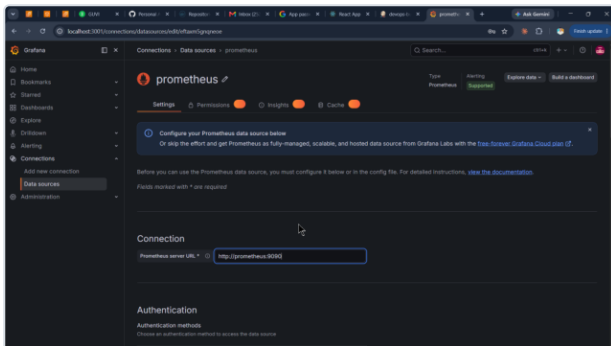
Grafana was configured against the local Prometheus instance (reached by its docker-compose service name prometheus, not localhost, since Grafana itself runs inside a container on that same compose network) and the community "Node Exporter Full" dashboard (ID 1860) imported.



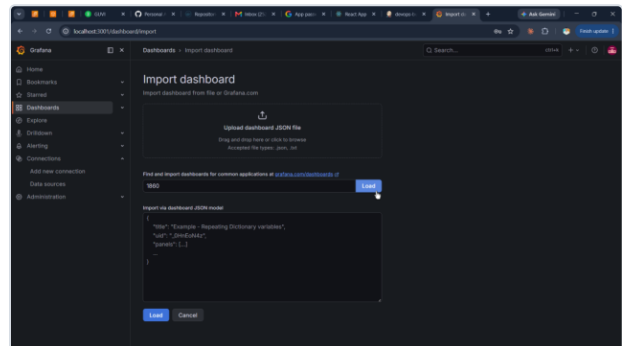
Screenshot 40 — Grafana login, reached through the SSH tunnel at localhost:3001.



Screenshot 41 — Prometheus data source configured at http://prometheus:9090.



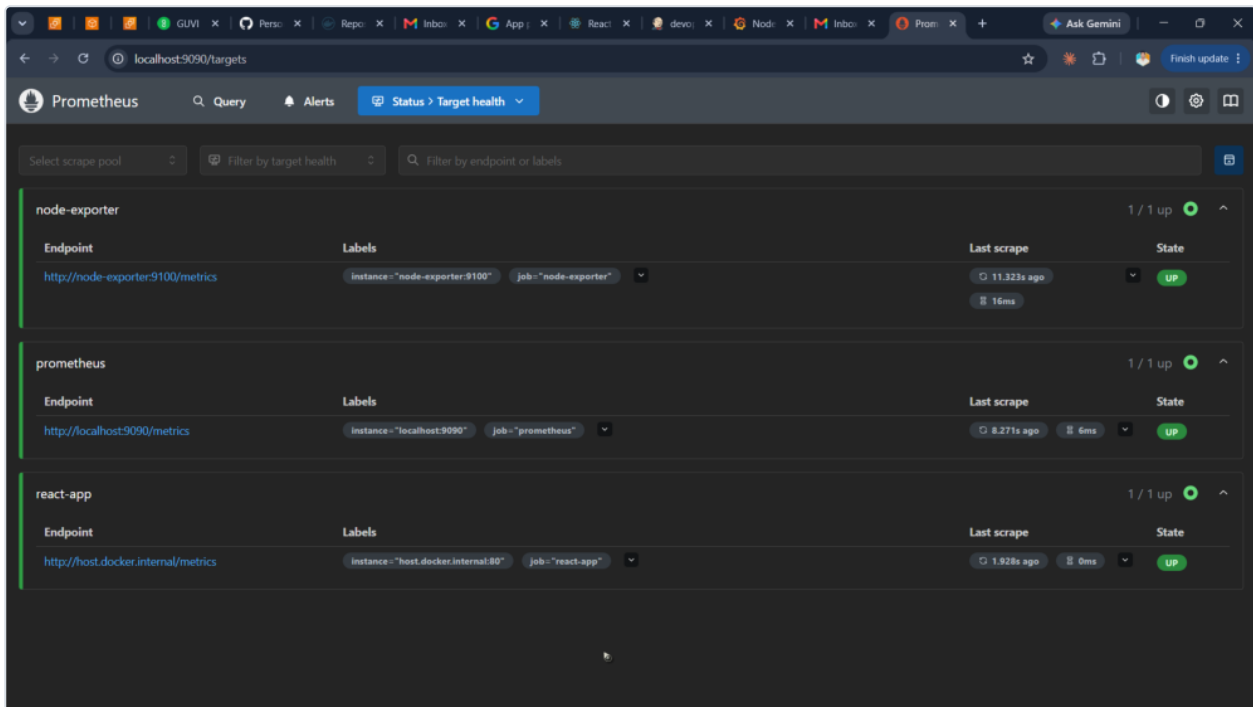
Screenshot 42 — "Successfully queried the Prometheus API" — data source verified live.



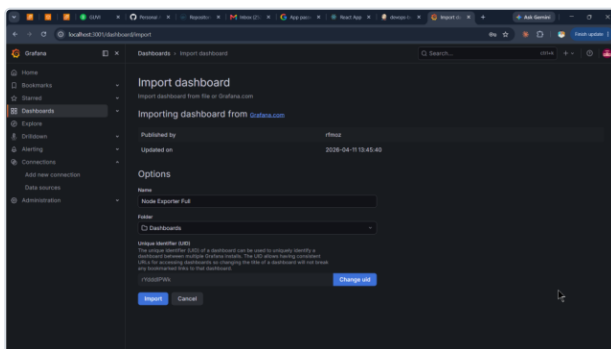
Screenshot 43 — Importing the "Node Exporter Full" community dashboard (ID 1860).

18. Monitoring Verification — Dashboards & Real Alerts

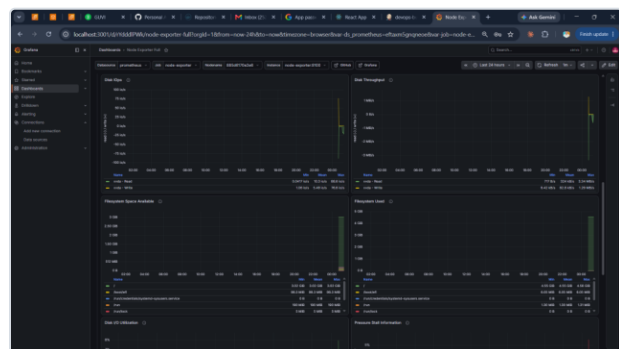
With the stack live, Prometheus's own target-health page confirmed all three scrape targets healthy — including the application itself, scraped via `host.docker.internal`, which required an explicit Linux host-gateway mapping in the compose file to resolve correctly on the real Ubuntu host (that mapping only matters on native Linux Docker; it is a no-op under Docker Desktop, which is why it did not surface as a problem during local testing).



Screenshot 44 — Prometheus target health: node-exporter, prometheus, and react-app all UP.



Screenshot 45 — Grafana "Node Exporter Full" dashboard rendering real, live metrics from the app instance — CPU, memory, disk.

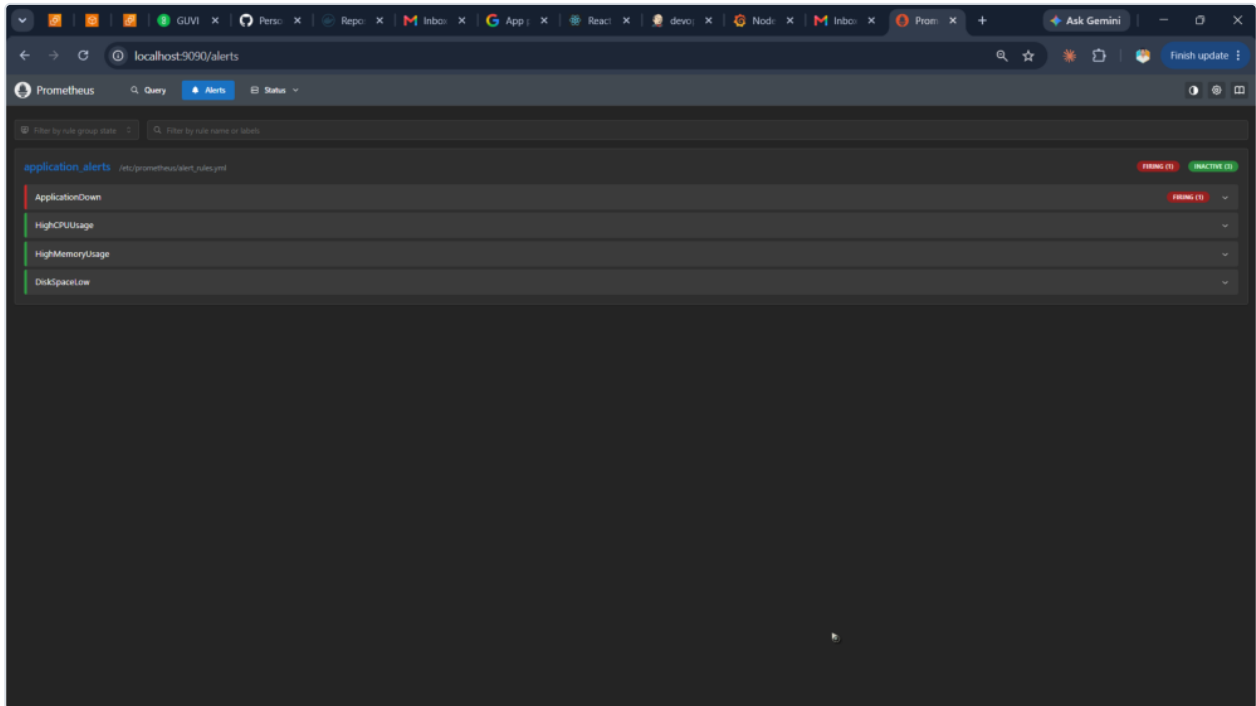


Screenshot 46 — Dashboard continued: disk I/O, filesystem usage, network — all populated with real data, not placeholders.

The down-alert requirement was tested for real, not just configured: the application container was stopped directly, and after the rule's one-minute threshold, Prometheus's own alert list showed ApplicationDown actively firing.

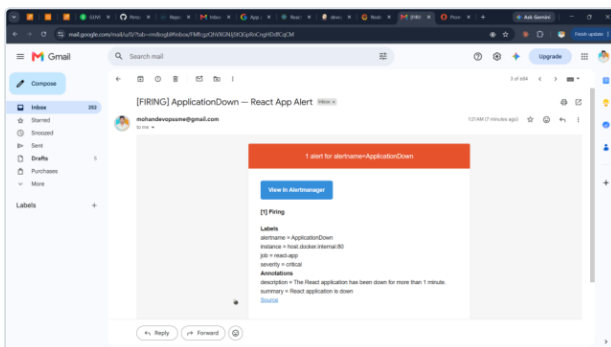
COMMANDS EXECUTED

```
$ docker stop devops-build-app
devops-build-app
```

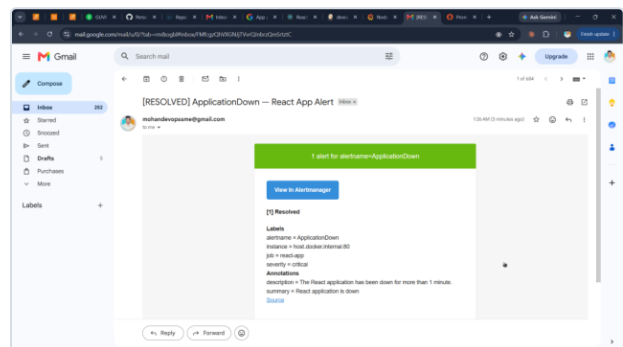


Screenshot 47 — Prometheus Alerts page: ApplicationDown FIRING, the other three rules correctly inactive.

Real emails were received at mohandevopssme@gmail.com from Alertmanager — both the critical application-down alert and a memory-usage warning that fired during the same window — and their corresponding resolved notifications once the container was restarted:



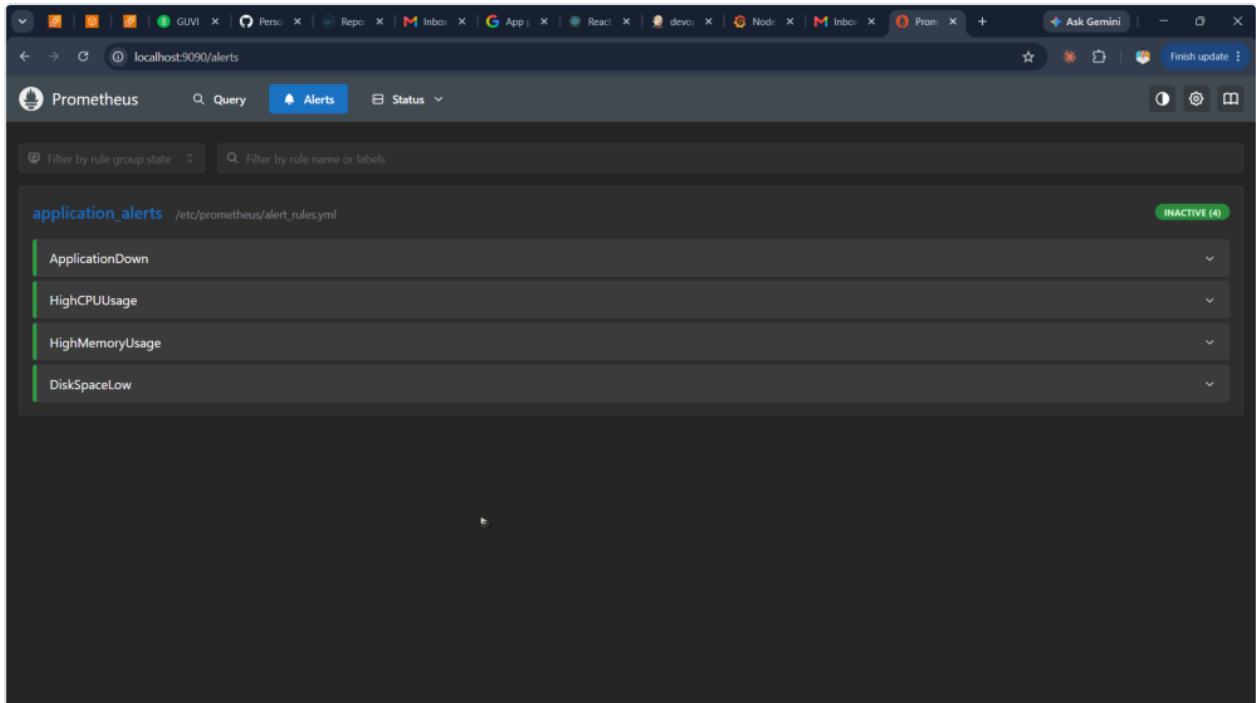
Screenshot 48 — "[FIRING] ApplicationDown — React App Alert", received in Gmail, severity critical.



Screenshot 49 — "[RESOLVED] ApplicationDown — React App Alert" after the container was restarted.

COMMANDS EXECUTED

```
$ docker start devops-build-app
devops-build-app
```



Screenshot 50 — Prometheus Alerts page back to all four rules INACTIVE after the application recovered.

19. Infrastructure Teardown

Once every requirement above was verified live — not just configured — both EC2 instances, both security groups, and the shared key pair were torn down via `aws/cleanup.sh` to stop billing. Verified clean afterward directly against the AWS API, not just trusted the script's own output:

COMMANDS EXECUTED

```
$ aws ec2 describe-instances --filters "Name=tag:Name,Values=devops-build-app,devops-build-jenkins" \
  --query 'Reservations[*].Instances[*].{Name:Tags[?Key==`Name`][0].Value,State:State.Name}' --output table
+-----+-----+
|      Name      |   State   |
+-----+-----+
| devops-build-app | terminated |
| devops-build-jenkins | terminated |
+-----+-----+

$ aws ec2 describe-security-groups --filters "Name=group-name,Values=devops-build-app-sg,devops-build-jenkins-sg" \
  --query 'SecurityGroups[*].GroupName' --output text
(empty)
```

Resource	Status
EC2 (app + Jenkins)	Terminated
Security groups (both)	Deleted
EC2 key pair	Deleted
DockerHub repositories (dev, prod)	Retained — durable record for submission, no AWS billing impact
GitHub webhook / Jenkins configuration	Retained — durable record for submission, no ongoing cost

Both instances confirmed terminated, no security groups remain, and the EC2 key pair was deleted from AWS.

20. Submission Summary

GitHub Repository: github.com/Mohan6201/guvi_tasks (branches: dev, main; path: Project-3/devops-build)

Deployed Site URL: <http://18.212.58.111> (verified live throughout; instance terminated post-verification per Section 19)

DockerHub Images: mohan809/dev (public), mohan809/prod (private)

Jenkins Job: devops-build-pipeline (Multibranch Pipeline)

Monitoring: Prometheus + Grafana + Alertmanager; real down-alert email verified